介入ベースの性能感度によるスプリアス特徴検出

要旨

機械学習モデルが訓練分布上で高い精度を示しながら、実際には因果的でない特徴（スプリアス特徴）に依存している事例は広く報告されている。候補特徴の発見は先行研究が扱っており、本研究の主眼はその後段で「その候補が本当に性能に効いているか」を確かめる確認手続きを定式化する点にある。本研究の PerformanceCheck は、訓練済みモデルが個別の特徴にどれだけ依存しているかを、入力に対する局所的な介入と性能変化の観測のみから定量化する手続きである。PerformanceCheck は、候補特徴を破壊的に置換する介入を適用し、性能低下量 Δ をその特徴の機能的必要性を表す近似指標として用いる。実験では合成データセットの GridWorld と ColoredMNIST、および実世界スプリアス相関を含む Waterbirds を用い、30 シードで評価した。GridWorld 10 次元では top-1 検出率 85/120 (71%, 95% CI [0.62, 0.78])、top-2 では 106/120 (88%, 95% CI [0.81, 0.93])、ColoredMNIST IRM では 102/120 (85%, 95% CI [0.78, 0.90])、Waterbirds では 18/30 (60%, 95% CI [0.42, 0.75]) を得た。同一の訓練済みモデルで比較した SHAP は、GridWorld では top-1 14/120 (12%, 95% CI [0.07, 0.19]) にとどまり、ColoredMNIST では本稿で採用した集約規則の下で 120/120 (100%, 95% CI [0.97, 1.00])、Waterbirds では 30/30 (100%, 95% CI [0.89, 1.00]) で背景候補を指摘した。さらに、候補発見の近似指標を前段に置く追加実験では、GridWorld と Waterbirds の双方で候補発見と確認の接続可能性を調べ、候補の存在を捉える指標と、実際に性能をどの程度損なうかを測る確認指標とが異なる役割を持つことを確認した。これらの結果は、特徴相関や符号化制約が強い条件では、性能ベースの介入テストが寄与度推定法とは異なる有用な診断情報を与えることを示唆する。

序論

機械学習モデルは、訓練分布のもとで高い予測性能を達成しても、実環境にデプロイされた際に背景や色など因果的でない特徴に依存し、分布シフトのもとで性能が低下する事例が繰り返し報告されてきた [Geirhos2020]。この問題はショートカット学習として整理されており、画像分類における背景バイアス、自然言語処理における表層パターン、強化学習における環境ダイナミクスの誤利用など、領域を横断して現れる。スプリアス特徴を訓練段階で抑制する手法として、不変リスク最小化 [Arjovsky2019]、群分布ロバスト最適化 [Sagawa2020]、反事実不変性 [Veitch2021] が提案されてきた。これらは強い帰納バイアスや群ラベルを要求し、訓練手続きそのものを変更する。

訓練後のモデルに対するスプリアス依存の診断は、これとは異なる課題である。診断側では、訓練手続きを再構成することなく、すでに学習済みのモデルがどの特徴に依存しているかを検査する必要がある。事後説明法のうち SHAP [Lundberg2017]、LIME [Ribeiro2016]、統合勾配 [Sundararajan2017] は局所的な特徴量寄与度を与えるが、これらの寄与度は出力分散への寄与や勾配の積分として定義され、「特徴を破壊したときに性能がどれだけ落ちるか」という運用上の関心と必ずしも一致しない。Adebayo らは、未知のスプリアス相関の検出という具体的な課題において、事後説明法が体系的に失敗しうることを示している [Adebayo2022]。Janzing らは、寄与度推定の背後にある因果的問題設定を整理し、観測同時分布のみに基づく寄与定義の限界を指摘している [Janzing2020]。

本研究で扱う問いは、特徴の重要度を出力分散や勾配の観点から定義するのではなく、性能の観点から直接定義した場合、発見済みの候補特徴に対する確認手続きとしてどの程度有効かというものである。PerformanceCheck は、候補特徴に対して破壊的な置換介入を施し、その前後でモデルの性能がどれだけ変化するかを測定する。この差分量を Δ と呼び、Δ が大きいほどモデル予測におけるその特徴の機能的必要性が

大きいと解釈する。介入は入力空間で行われるため、モデル内部の勾配や Shapley 値の近似を必要とせず、ブラックボックスとして扱える。なお、候補特徴の自動列挙自体は Neuhaus ら [Neuhaus2023]、Eyuboglu ら [Eyuboglu2022]、Lapuschkin ら [Lapuschkin2019] などが別途扱っており、本研究の新規性はそれらの代替ではなく、出力された候補を受け取り、その候補が本当に性能に効いているかを確認する後半のステップを定式化した点にある。実務上の流れとしては、まず前段の候補発見法が疑わしい特徴候補を返し、次に PerformanceCheck がその候補を壊したときの性能差分を測り、最後にその差分を人間が修正優先度の判断材料として読む、という三段の手順を想定する。

PerformanceCheck の動作を、合成データセットと実世界データセットの双方で検証した。GridWorld は 10 次元の特徴ベクトルを持ち、そのうち 1 次元がスプリアス特徴 c、2 次元が因果特徴である agent_x と agent_y、残る 7 次元がノイズである。スプリアス特徴と教師信号の相関確率 spurious_p を 0.0 から 0.3 まで変化させ、各設定で 30 シードを評価した。ColoredMNIST はデジット形状と色に対して相関構造を導入したベンチマークで、不変リスク最小化を訓練手続きとして適用した訓練済みモデルを用いる。Waterbirds は陸鳥と水鳥の分類において背景がスプリアス特徴として作用する実世界寄りのベンチマークである [Wah2011, Sagawa2020]。

得られた結果は、PerformanceCheck が GridWorld では SHAP より高い検出率を示すこと、ColoredMNIST では SHAP が満点を出すものの、本稿で採用した集約規則の下ではその満点がデータ符号化に起因する構造的アーティファクトとなり、訓練済みモデルの実際の依存関係を反映しないことを示している。Waterbirds では、SHAP が 30/30 シードで背景を候補として指摘する一方で、PerformanceCheck は 18/30 シードで背景介入の方が前景介入より大きな性能差分を与えることを示し、候補指摘と機能的必要性の確認とが別の役割を持つことを示した。これらの観察から、PerformanceCheck は少なくとも本研究で扱った条件において、性能観点での依存確認に対して有用な補完情報を与えると考えられる。

本研究は、スプリアス候補の発見と依存確認という二段階の流れのうち、後者の確認ステップを PerformanceCheck として定式化することで、既存の候補発見法を受け取って発見から診断までを一貫させる枠組みを与える。主たる貢献は、訓練済みモデルに対して入力空間での破壊的介入と性能差分のみから特徴依存の機能的必要性を近似測定する点と、GridWorld・ColoredMNIST IRM・Waterbirds の三つの設定でその有効性を SHAP 比較を通じて示す点にある。補助的に、GridWorld と Waterbirds の追加実験では、簡単な候補発見の近似指標、専用スコア、CLIP、自己教師あり表現を前段に置いたときの接続可能性を調べ、候補の存在を返す前段と機能的必要性を返す後段の役割分担を具体化した。さらに、ColoredMNIST では、本稿で採用した SHAP 集約規則の下で見かけ上の高検出率が、R チャネルと G チャネルの符号化非対称性から生じる構造的アーティファクトであることを、各シードの特徴量寄与度と比例関係の観点から明らかにした。

関連研究

スプリアス相関に関する既存研究は、訓練アルゴリズムによる対処と、事後的な診断・寄与度推定の二つに大別できる。不変リスク最小化 [Arjovsky2019]、群分布ロバスト最適化 [Sagawa2020]、反事実不変性 [Veitch2021] はいずれも訓練手続きを変更し、スプリアス特徴への依存を抑制する枠組みである。Zhou ら [Zhou2021] はこの系譜を分布シフトの観点から整理している。これらに対し本研究は、既存の学習済みモデルを対象とする事後診断を扱う。

事後的な寄与度推定の系譜では、SHAP [Lundberg2017] が Shapley 値の協力ゲーム理論を機械学習モデルの説明に適用し、特徴ごとの平均的寄与を出力分散の分解として定義する。LIME [Ribeiro2016] は局所的に線形代理モデルを学習することで近似的な特徴重要度を得る。統合勾配 [Sundararajan2017] は基準点から入力までの経路積分により公理的に特徴量寄与度を定める。これらの寄与度推定法は、モデル予測の説明という枠組みでは整合的であるが、スプリアス特徴の検出を目的としたとき、観測分布における特徴の出力分散寄与と、特徴を実際に破壊したときの性能感度との間にギャップが生じうる。本研究はこのギャップを実験的に示す。

Adebayo らは、未知のスプリアス相関に対して事後説明法が信頼性のある検出能力を持たない場合があることを実証しており [Adebayo2022]、Janzing らは、寄与量を観測同時分布のみから定義する立場と、介入を伴う因果的立場との差異を整理している [Janzing2020]。

スプリアス特徴を含むベンチマークとしては、ColoredMNIST が広く用いられ、デジット形状と色との相関を意図的に操作することで不変学習の評価を可能にする [Arjovsky2019]。Waterbirds は CUB-200-2011 [Wah2011] の鳥画像を Places365 由来の背景に合成して構成され、陸鳥と水鳥の分類において背景がスプリアス特徴として作用する [Sagawa2020]。Singla らは ImageNet クラスごとの「コア」領域と「スプリアス」領域を人手で同定し、これに基づくリスク最小化を提案している [Singla2022]。これらのデータセットはスプリアス領域が空間的に局在しているため、入力空間での介入が比較的明瞭に定義できる。本研究の Waterbirds 実験では、GrabCut で生成した前景マスクに基づく背景置換と、前景に対するノイズ付加を、それぞれ独立した介入として実装した。

スプリアス特徴の**自動発見**を目的とした研究も進んでいる。Neuhaus ら [Neuhaus2023] は事前学習済み分類器の最終隠れ層活性化に Neural PCA を適用し、Eyuboglu ら [Eyuboglu2022] は CLIP 埋め込みと混合モデルで系統的失敗スライスを発見し、Lapuschkin ら [Lapuschkin2019] は LRP 帰属マップのスペクトルクラスタリング（SpRAy）で一貫した帰属パターンを抽出し、Sohoni ら [Sohoni2020] は誤り分布からスプリアス相関を持つサブグループを群ラベルなしで発見する。これらの手法が「どの特徴がスプリアス候補か」を返すのに対し、本研究の PerformanceCheck はその候補を受け取り、依存の有無を性能差分として確認する後段ステップとして機能する。

PerformanceCheck は、入力空間における破壊的介入と性能差分という二要素のみを使用する点で、特徴空間に閉じた寄与度推定法とは異なる。未知のスプリアス相関に対する事後説明法の限界を体系的に検証したものとして Adebayo ら [Adebayo2022] が近接するが、本研究は破壊的介入による性能差分を直接の指標として定式化し、合成と実世界の両ベンチマークで定量的比較を行う点に違いがある。

手法

PerformanceCheck は、モデル f と評価データ D、候補特徴集合 F を入力として受け取り、各特徴 i ∈ F に対する性能感度 Δ_i を出力する。実運用上の出力は、この Δ に基づいて並べた候補の順位リストであり、Δ 自体は機能的必要性の近似値として読み、手法全体の役割は候補発見の後段に置く確認と優先順位づけにある。手法はモデルを再訓練しないため、診断としての適用負荷が低い。本節ではまず PerformanceCheck の定式化を述べ、次に介入演算子の設計、判定基準、ベースライン手法 SHAP との対応関係を順に示す。 本稿では、この手法を PerformanceCheck と呼ぶ。

性能感度 Δ の定式化

評価データ D 上のモデル f の性能を P(f, D) と表記する。分類問題では精度を、回帰や強化学習では適切なスカラー指標（平均報酬など）を用いる。候補特徴 i ∈ F に対する破壊的介入 T_i を定義し、介入後のデータ T_i(D) を構成する。T_i は特徴 i の値を、D における周辺分布からのランダムリサンプリング、定数置換、外部マスクによる置換などの手段で破壊する。性能感度は次式で定める。

Δ_i = P(f, D) − P(f, T_i(D))

Δ_i が大きい特徴は、モデル予測においてその特徴の機能的必要性が大きいことを示す。Δ_i は性能の差分として定義されるため、確率出力やソフトマックス分布を経由しない点で、SHAP のような出力分布レベルの寄与度とは異なる。本研究の介入 T_i は、Pearl の do 演算 [Pearl2000] に着想を得た実用的な介入近似として、対象特徴の情報を破壊しつつ他特徴を可能な限り保持する操作として設計する。Janzing ら [Janzing2020] が

論じるように、観測同時分布に基づく寄与量は、特徴間相関が強い場合に「出力分散への寄与」と「予測の操作的必要性」を一致させないことがある。Adebayo ら [Adebayo2022] の指摘と整合的に、本研究は因果性そのものの証明を目的とせず、「その特徴を壊したときに予測能力がどれだけ失われるか」を操作的に測定する。したがって Δ は、因果効果そのものではなく、依存候補を性能に効く順へ並べるための機能的必要性の近似指標として解釈される。

介入演算子の設計

GridWorld 設定では、特徴ベクトル x ∈ R^10 のうち i 次元目を、訓練分布から経験的に推定した周辺分布から独立にサンプリングして置換した。これにより、当該特徴とラベルとの関係は破壊されるが、他特徴の周辺分布は維持される。ColoredMNIST IRM では、色チャネルに対する介入として R チャネルと G チャネルを乱数で置換する操作を採用した。digit の構造特徴に対する介入は、形状を保ったまま色を中立化する操作で実装した。Waterbirds では二種類の介入を独立に評価した。background_replace は、GrabCut により生成した前景マスクの外側を、同一クラスの候補群からサンプルした他画像の背景で置換する class-consistent な操作である。foreground_noise は、前景マスク内側に標準偏差 0.3 のガウスノイズを加える操作である。LunarLander では、状態ベクトルに対して候補特徴次元を一様乱数で置換する。

介入演算子は、特徴を「壊す」際に他の特徴情報をできるだけ保持することが望ましい。実装上は、Waterbirds の前景マスクのように特徴の空間的局在を利用できる場合と、GridWorld のように特徴次元が直接識別できる場合とで実装が分かれる。

介入演算子の妥当性条件として、T_i は (a) ターゲット特徴 i の情報を破壊し、(b) 他特徴の周辺分布を可能な限り保持することが望ましい。条件 (a) は候補依存の検出感度を確保し、条件 (b) は他特徴への交絡を避けるために必要である。GridWorld の周辺リサンプリングは両条件を満たす。Waterbirds の background_replace は、前景マスクで前景情報を保ちながら、同一クラス候補から背景のみを置換することで条件 (b) を実用的に近似する。一般に、完全な条件 (b) の充足は困難であり、介入の質が Δ の信頼性に影響する点は適用時に留意すべきである [Janzing2020]。同一クラス候補を用いる設計は、鳥種と背景が同時に不自然に崩れる強い OOD 介入を避ける利点を持つ一方で、クラスに結び付いた背景手掛かりを部分的に温存したり、クラス非依存の背景バイアスを弱めてしまったりする可能性もある。

この観点から、本稿では「対象特徴だけを狙って壊す構造保持型介入」と「入力全体を強く乱してしまう破壊型コントロール」を区別する。たとえば画像全体への強いノイズ付加や意味を保たない大域的置換は、条件 (a) を満たしても条件 (b) を損ないやすく、OOD 由来の性能低下と切り分けにくい。これに対し、Waterbirds の背景置換や前景マスク内ノイズは、完全ではないにせよ、対象特徴の破壊と非対象特徴の保持を両立させる近似として設計されている。

検出判定基準

特徴 i が「依存検出された」と判定する主基準は、Δ_i が候補集合内で上位に位置するかどうかとした。したがって本稿の主指標は top-k inclusion に基づく recall@k であり、主たる運用形は修正優先順位づけに用いる全候補のランキング提示である。実験では per-seed で Δ ランキングを計算し、対象候補が top-k に入るかどうかを判定した。GridWorld と ColoredMNIST では 10 次元ないし少数候補のランキングから recall@k を集計し、GridWorld の p-sweep では spurious_p ∈ {0.0, 0.1, 0.2, 0.3} の各値を 30 シードずつ、合計 120 試行で評価した。Waterbirds のように候補が背景と前景の二領域に事実上集約される設定では、pairwise な比較は recall@1 と同値な補助表現として用いる。

ColoredMNIST IRM の高 spurious_p 領域では、訓練済みモデルが構造特徴に依存を切り替える設計となっており、判定基準もそれに応じて structure 候補が color 候補より上位にランクされることを要求する。

SHAP ベースラインの設計

GridWorld 比較では SHAP の KernelExplainer を使用し、訓練分布から背景サンプルを取って各テスト点の Shapley 値を推定した。スプリアス c と因果 agent_y の Shapley 値の絶対値を比較し、$|\varphi_c| > |\varphi_{agent_y}|$ のとき検出と判定した。ColoredMNIST IRM 比較では GradientExplainer を採用し、SHAP に対してより有利な解釈を与えるため、R チャネルと G チャネルの非対称性を活用した集約を施した。具体的には、color_attr = mean(|shap_R − shap_G|)、struct_attr = mean(|shap_R + shap_G|) / 2 と定義した。Waterbirds 比較では、前景マスクに基づき SHAP 値を空間集約し、shap_bg と shap_fg の比較で判定した。

ColoredMNIST における SHAP 集約は、データ符号化を考慮した結果でも、なお構造的制約を受ける。当該データセットでは赤い digit の場合 R に digit 強度・G は 0、緑の digit の場合 G に digit 強度・R は 0、と排他的に符号化されている。あるサンプルにおいて R チャネルが活性であれば G チャネルは恒等的に 0 であるため、shap_G ≈ 0 となり、本稿で採用した集約規則の下では color_attr ≈ |shap_R|、struct_attr ≈ |shap_R| / 2 に縮退する。結果として color_attr ≈ 2 × struct_attr が、訓練済みモデルの実際の挙動とは独立に成立してしまう。Method 節ではこの集約式自体を定義に含め、Results 節と Discussion 節でその帰結を扱う。

訓練済みモデルの構成

GridWorld では、30 シードそれぞれで spurious_p に応じた相関を持つデータセットを生成し、多層パーセプトロンを訓練して訓練精度が十分に高い時点で停止した。ColoredMNIST では、IRM のオリジナル定式化に倣った損失関数で訓練し、label_noise_p=0.25 の設定で固定した。Waterbirds では、ImageNet 事前学習済み ResNet-18 [He2016] の特徴抽出層を凍結し、最終層を Linear(512, 2) に置換して 10 エポック fine-tune した。すべての実験で、PerformanceCheck と SHAP 比較は同一の訓練済みモデル・同一シードで実施した。

実装と乱数管理

各実験では Python の numpy と PyTorch における乱数シードを統一して制御し、データ生成・モデル初期化・介入時の置換サンプリングがシードに対して再現的に動作することを確認した。SHAP の計算では、shap ライブラリの KernelExplainer と GradientExplainer をそれぞれ規定の設定で使用した。

実験

評価設定は四つに分かれる。第一に、GridWorld 10 次元 p-sweep では、スプリアス特徴 c との相関確率 $\text{spurious_p} \in \{0.0, 0.1, 0.2, 0.3\}$ を 30 シードずつ評価し、PerformanceCheck の検出率を p の関数として測定した。第二に、ColoredMNIST IRM p-sweep では、label_noise_p を 0.25 に固定したうえで spurious_p を同範囲で変化させ、30 シードずつ評価した。第三に、PerformanceCheck と SHAP の比較を、GridWorld 10 次元と ColoredMNIST IRM で実施した。同一の訓練済みモデル・同一シードのもとで両手法を計算し、検出率を直接比較した。第四に、Waterbirds では、waterbird_complete95_forest2water2 サブセットで凍結 ResNet-18 を fine-tune した訓練済みモデルに対し、background_replace と foreground_noise の二種類の介入を行い、PerformanceCheck と SHAP の検出率を 30 シードで測定した。なお GridWorld p-sweep と Δ vs SHAP 比較は同一の訓練済みモデル・同一シードで実施しており、両表記は同一の実行結果に基づく。

各ベンチマークで「正解」と見なすものは同一ではない。GridWorld では真のスプリアス特徴 c がランキング上位に入るか、ColoredMNIST では color 候補と structure 候補のどちらが上位か、Waterbirds では背景候補と前景候補のどちらがより大きな性能差分を生むかを評価する。したがって本稿の比較は、候補集合が既知の設定での確認能力を主に見ており、候補自動発見そのものの性能比較ではない。

ベンチマーク	候補集合	正解の意味	主指標
GridWorld	10 次元特徴（c, agent_x, agent_y, noise×7）	真のスプリアス特徴 c が上位候補に入る	recall@k
ColoredMNIST	color 候補, structure 候補	訓練済みモデルの依存先に応じて正しい候補が上位に入る	recall@1
Waterbirds	背景候補, 前景候補	背景介入と前景介入のうち、より大きな性能差分を生む候補が上位に入る	recall@1

統計検定の位置づけも分けて扱う。GridWorld と ColoredMNIST における Δ と SHAP の paired McNemar 検定は主比較に対する確認的検定として用い、Waterbirds の repeated-intervention 平均差に対する符号検定、および候補発見と確認の整合率評価は探索的な補助分析として扱う。多重比較補正は行っていないため、探索的分析の p 値は効果の大きさとあわせて解釈する。

GridWorld の特徴ベクトルは 10 次元で、スプリアス特徴 c × 1、因果特徴 agent_x と agent_y × 2、ノイズ × 7 から構成される。主評価では c が全候補ランキングの上位 k に入るかどうかで判定し、Δ_c > Δ_agent_y は補助的な二候補比較としてのみ用いる。ColoredMNIST IRM の評価では、color 候補と structure 候補のランキングで判定する。Waterbirds の評価では、背景と前景それぞれの介入による Δ を比較し、Δ_bg > Δ_fg を背景候補が上位 1 位に入る場合と等価な recall@1 の表現として用いる。

LunarLander 環境では、訓練済み方策に対して候補スプリアス特徴を破壊する介入を試みた。本論文では Results の主張対象から分離し、Appendix の適用境界事例として扱う。GridWorld / ColoredMNIST / Waterbirds は 30 シードで評価した。

この論文で確かめたいのは、候補発見の後段に置く確認手続きとして PerformanceCheck がどの程度機能するかである。その確認を五段階に分けて数値実験で行う。第一に GridWorld の相関掃引で、相関強度に応じた挙動を確かめる。第二に ColoredMNIST IRM で、依存先の切り替えを追跡できるかを確かめる。第三に GridWorld と ColoredMNIST の比較で、SHAP とどのように異なる情報を返すかを確かめる。第四に Waterbirds で、実世界寄りの背景依存に対してどの程度確認が効くかを確かめる。第五に候補発見との接続実験で、前段が返した候補を後段でどこまで整理できるかを補助的に確かめる。

結果

以下では、上の五段階に沿って PerformanceCheck がどのような情報を返すかを整理する。

GridWorld の相関掃引

GridWorld の相関確率掃引実験（n=30 シード）では、PerformanceCheck はスプリアス相関が弱い領域で高い検出率を維持し、相関が強くなるにつれて段階的に性能が低下した。これは、p の上昇に伴ってモデルがスプリアスと因果の両方を併用する状態へと遷移し、介入による相対差が縮まるためと考えられる。詳細な値は後掲の比較表に示す通りであり、上位候補まで見ると top-1 のみの場合より安定した検出が得られる。

PerformanceCheck はすべての特徴に対して Δ_i を計算するため、最大値を与える一つだけでなく上位 k 件を候補として提示する使い方も自然に定義できる。上位 k 件に真の候補が含まれるかという観点では、Δ は候補特徴を少数の上位候補に絞り込む情報を与え、最終判断をドメイン知識に委ねる使い方が実用的と考えられる。

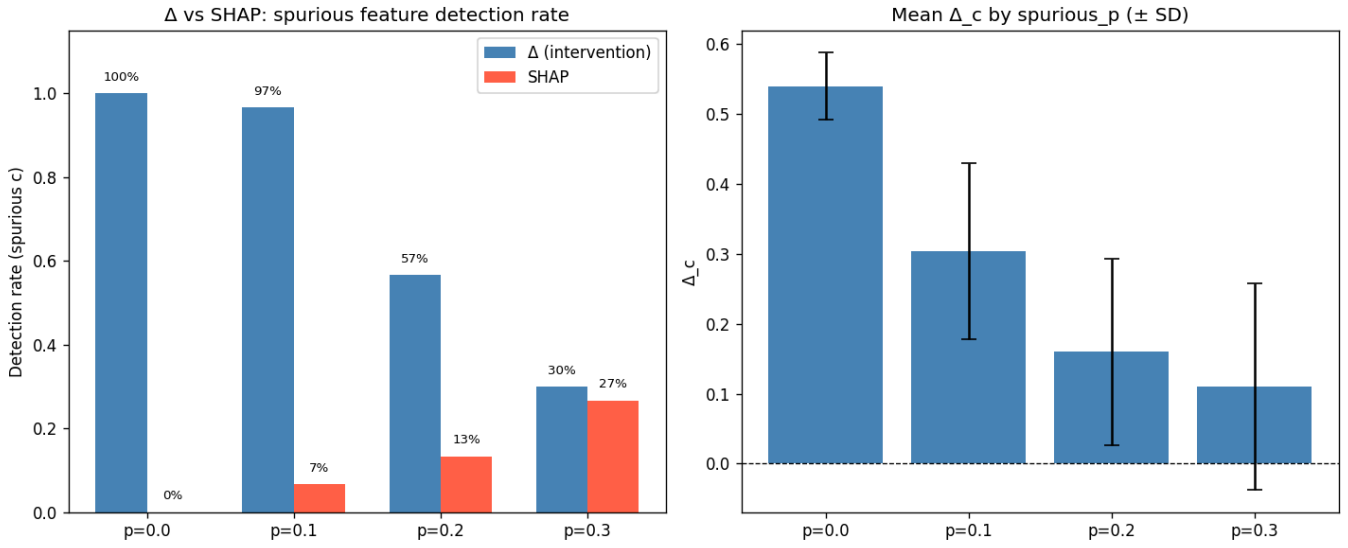
ColoredMNIST IRM の相関掃引

ColoredMNIST IRM p-sweep（n=30 シード）でも類似のパターンが観察された。相関が強くなるにつれて Δ の検出率は下がるが、GridWorld よりはやや粘る傾向があり、これは IRM による正則化の影響と整合的である。詳細な数値は後段の比較表に示す通りであり、特に p=0.3 で構造依存への切り替えが顕在化する。

Δ と SHAP の比較

GridWorld 10 次元での Δ と SHAP の比較（n=30 シード）では、両手法の差が明確に現れた。Δ はスプリアス特徴を上位候補に押し上げられる一方、SHAP は出力分散寄与に引っ張られるため検出が伸びなかった。対応のある McNemar 検定でもこの差は有意であり、top-1 では `p = 8.38 × 10^-16`、top-2 でも `p = 2.04 × 10^-4` であった。

spurious_p	Δ@1	Δ@2	SHAP@1	SHAP@2
0.0	30/30 (100%, 95% CI [0.89, 1.00])	30/30 (100%, 95% CI [0.89, 1.00])	0/30 (0%, 95% CI [0.00, 0.11])	0/30 (0%, 95% CI [0.00, 0.11])
0.1	29/30 (97%, 95% CI [0.83, 0.99])	29/30 (97%, 95% CI [0.83, 0.99])	2/30 (7%, 95% CI [0.02, 0.21])	3/30 (10%, 95% CI [0.03, 0.26])
0.2	17/30 (57%, 95% CI [0.39, 0.73])	25/30 (83%, 95% CI [0.66, 0.93])	4/30 (13%, 95% CI [0.05, 0.30])	9/30 (30%, 95% CI [0.17, 0.48])
0.3	9/30 (30%, 95% CI [0.17, 0.48])	17/30 (57%, 95% CI [0.39, 0.73])	8/30 (27%, 95% CI [0.14, 0.44])	9/30 (30%, 95% CI [0.17, 0.48])
合計	85/120 (71%, 95% CI [0.62, 0.78])	106/120 (88%, 95% CI [0.81, 0.93])	14/120 (12%, 95% CI [0.07, 0.19])	21/120 (18%, 95% CI [0.12, 0.25])



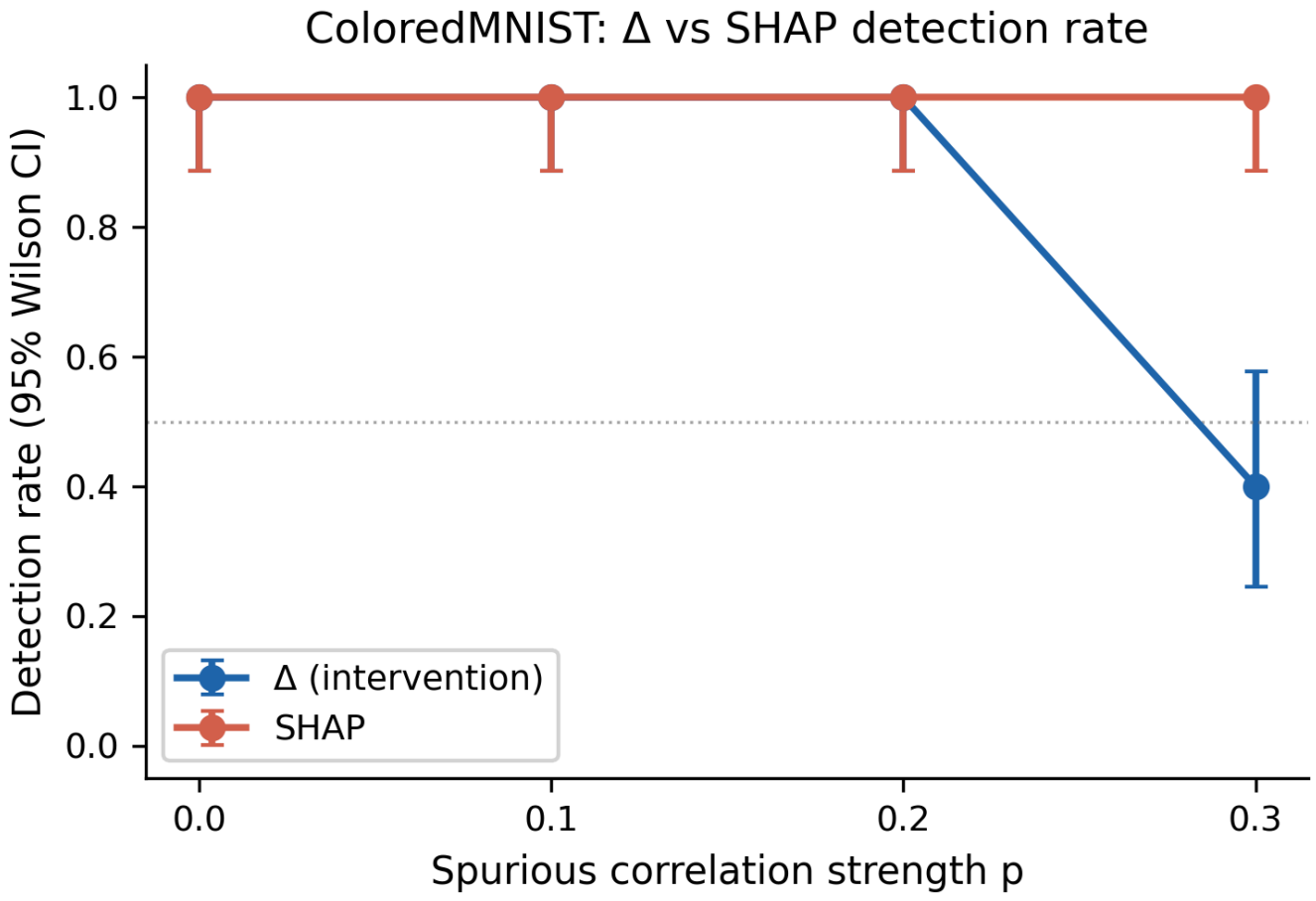
SHAP の上位 k 件への包含率は低相関領域でもほとんど改善せず、SHAP が構造的に c を上位に入れにくいことを示している。これに対し PerformanceCheck では、相関が弱い領域では top-1 でも安定し、相関が中程度まで強まった場合でも上位候補の絞り込みとしては有効性を保つ。

ColoredMNIST IRM での比較（n=30 シード）は、見かけ上は SHAP に有利な結果を示すが、その内訳には注意が必要である [Arjovsky2019]。本稿で採用した R-G 非対称集約の下では、SHAP の満点はデータ符号化と集約規則の組み合わせによる構造的アーティファクトに由来する。paired な McNemar 検定では、Δ と SHAP の判定差は `p = 7.63 × 10^-6` で有意だった。

この性質は p=0.3 における各シードの挙動に表れる。構造依存へ切り替わったシードでは Δ がそれを追跡し、残るシードでは色依存として検出される。

例示シード（p=0.3、n=5）	delta_hue	delta_struct	Δ 判定	SHAP 判定
1	0.081	0.162	NG（構造切替を検出）	OK
2	0.066	0.124	NG（構造切替を検出）	OK
0	0.061	0.072	NG（構造切替を検出）	OK
4	0.088	0.081	OK	OK
5	0.098	0.073	OK	OK

spurious_p	Δ 検出	SHAP 検出
0.0	30/30 (100%, 95% CI [0.89, 1.00])	30/30 (100%, 95% CI [0.89, 1.00])
0.1	30/30 (100%, 95% CI [0.89, 1.00])	30/30 (100%, 95% CI [0.89, 1.00])
0.2	30/30 (100%, 95% CI [0.89, 1.00])	30/30 (100%, 95% CI [0.89, 1.00])
0.3	12/30 (40%, 95% CI [0.25, 0.58])	30/30 (100%, 95% CI [0.89, 1.00])
合計	102/120 (85%, 95% CI [0.78, 0.90])	120/120 (100%, 95% CI [0.97, 1.00])

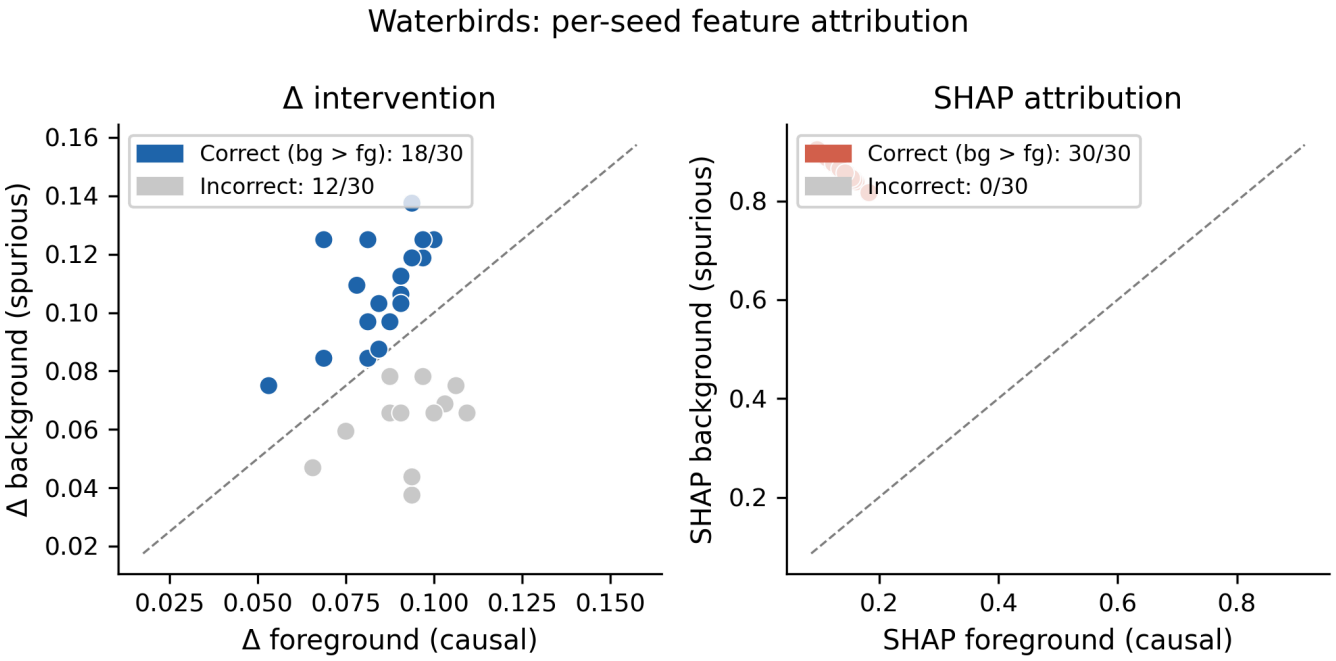


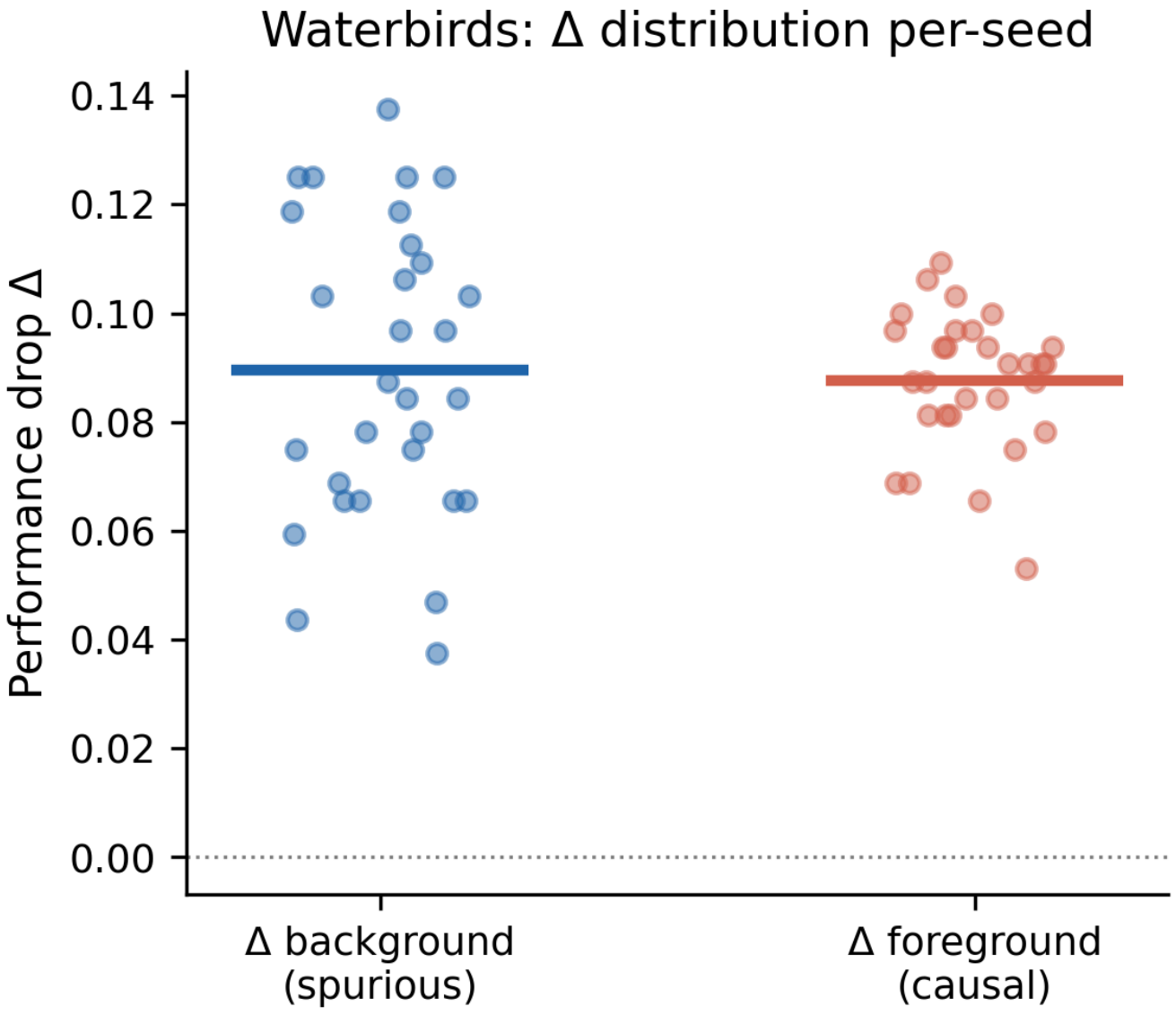
SHAP は同じ 3 シードで、本稿で採用した集約規則の下では color_attr = 2 × struct_attr の比例関係が集約式からほぼ自動的に生じるため、構造依存への切り替えを追従できず一律に色依存と報告する。 判定基準上は

SHAP が「正解」と見えても、これは訓練済みモデルの依存先と判定結果が一致したのではなく、データ符号化が SHAP 集約の比例関係を強制した結果である。

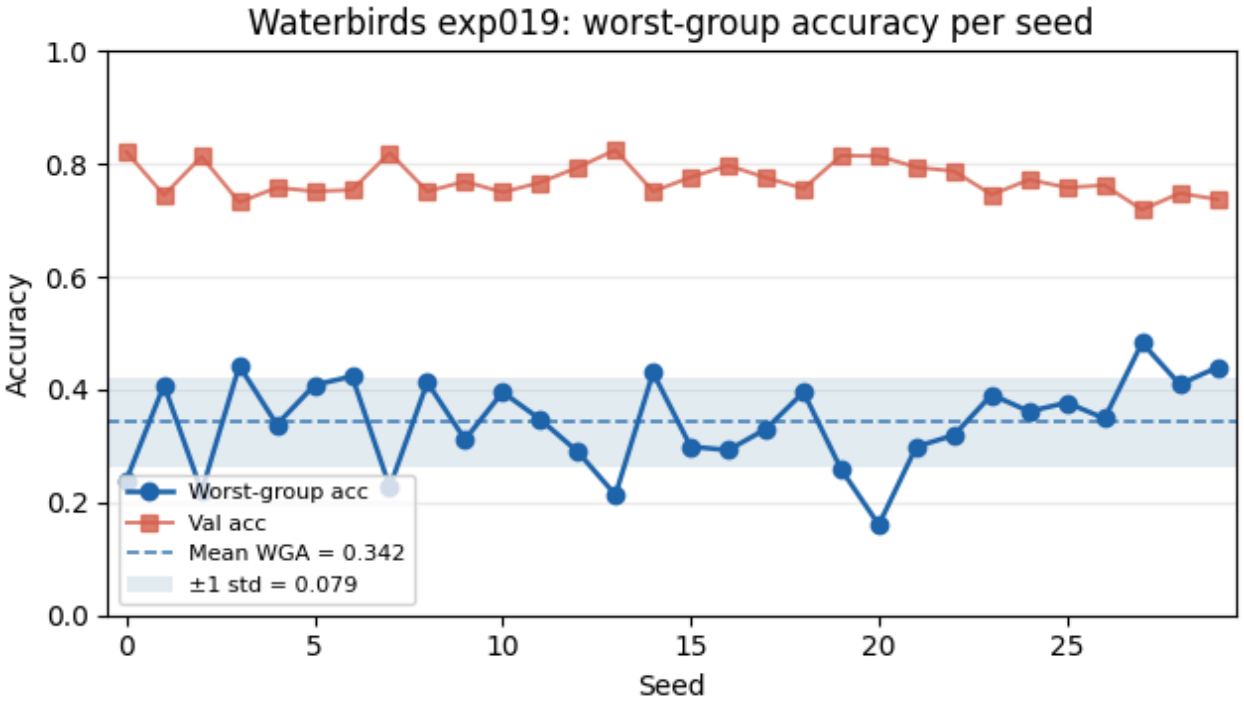
Waterbirds における確認

Waterbirds では、PerformanceCheck は 18/30 シード (60%, 95% CI [0.42, 0.75]) で背景を前景より上位に置き、SHAP は 30/30 シード (100%, 95% CI [0.89, 1.00]) で背景候補を指摘した [Wah2011, Sagawa2020]。したがって SHAP は背景候補の指摘には敏感だが、PerformanceCheck はその中で実際に大きな性能差分を伴う背景依存をより保守的に選別している。加えて、per-seed の連続差分 `Δ_bg - Δ_fg` の平均は 0.0018 で、95% CI は [-0.0089, 0.0124] であった。すなわち背景優位のシードが多数派ではあるものの、その差は一部のシードで境界的であり、18/30 という値は一律の失敗ではなく、モデル差・介入差・厳密な recall@1 定義が重なった混合的な結果として読むのが適切である。

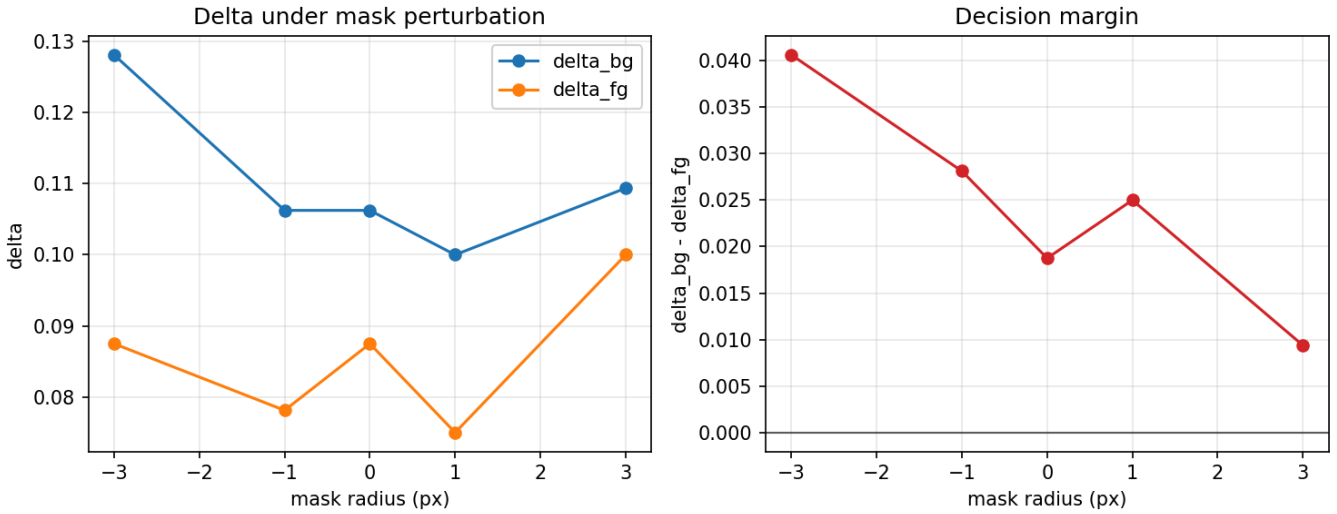




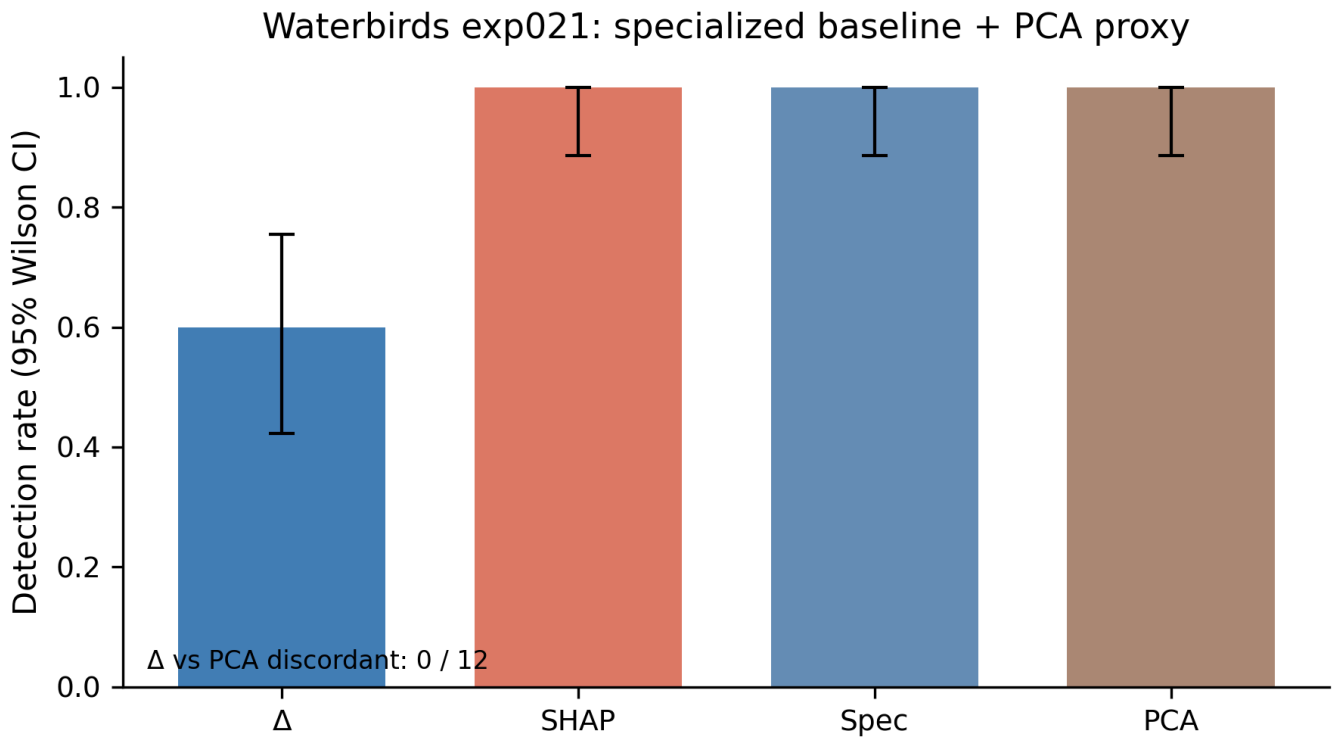
worst-group accuracy の再集計では、平均精度だけでは見えにくい群間不均衡が確認された。



この傾向が介入実装の細部だけで決まっていないかを確かめるため、補助アブレーションとしてマスク感度も評価した。1 seed 固定の感度分析では、軽微なマスク摂動に対して結論が安定し、GrabCut の小さなズレがただちに結論を反転させないことが確認された。加えて 30 シードの半径スイープでも、背景優位率は半径 -3 で 24/30 (80%, 95% CI [0.63, 0.90])、半径 0 で 17/30 (57%, 95% CI [0.39, 0.73])、半径 3 で 9/30 (30%, 95% CI [0.17, 0.48]) となり、平均 Δ_bg と平均 Δ_fg もマスク半径に対して連続的に変化した。したがって、マスク品質は結果の大きさと背景優位の出やすさには影響するものの、基準半径近傍で突然別の結論へ飛ぶというより、確認結果の強さを連続的に変える要因として働く。



Waterbirds の専用ベースライン比較では、保存済み checkpoint を用いて専用スコアを生成した。Spurious Score に基づく近似指標でも、隠れ層 PCA に基づく近似指標でも、背景スプリアスを示す傾向が一貫して観察された。これは、表現空間上の近似指標は候補の存在を捉えやすい一方で、介入後の必要性とは別軸であることを示している。



この比較では、PerformanceCheck は背景依存を検出するシードとしないシードに分かれ、30 シード中 12 シードで SHAP / Spec / PCA と不一致になった。一方、SHAP / Spec / PCA は一貫して背景スプリアスを指摘した。さらに、各シードで 300 回の介入を平均した `bg_mean_delta - fg_mean_delta` は 30 シードすべてで正であり、両側符号検定でも有意だった (`p = 1.86 × 10^-9`)。ここで重要なのは勝敗そのものではなく、候補の存在を拾う指標と、実際に性能を壊すかどうかを見る確認指標とで、測っている量が異なることである。代表的な不一致シードの個別スコアを以下に示す。

シード	PerformanceCheck 判定	PCA 判定	Δ_bg	Δ_fg	bg_prob_score	pca_score
1	NG	OK	0.069	0.103	0.383	0.134
3	NG	OK	0.059	0.075	0.380	0.205
5	NG	OK	0.066	0.109	0.333	0.153

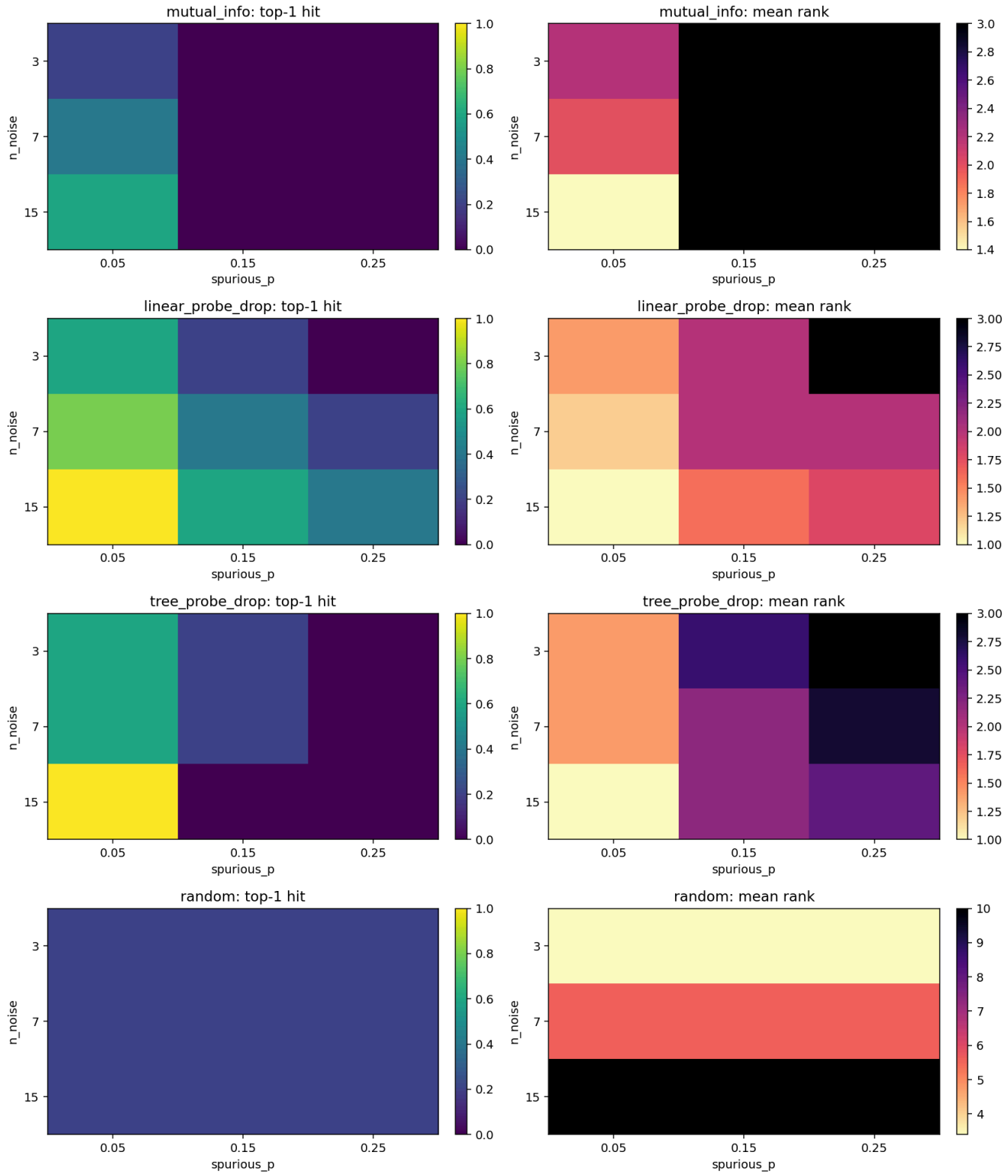
これらの不一致シードにおいて、専用手法（PCA 等）は特徴空間内にスプリアス成分が「存在すること」を捉えているが、PerformanceCheck は介入後の性能が前景介入に比べて落ちていない事実から、モデルが不変性を獲得していると保守的に判定している。したがって、Spec/PCA は表現空間上の近似指標、PerformanceCheck は入力介入に対する機能的必要性の近似指標として読むのが適切である。

例示シード（n=5）	val_acc	delta_bg	delta_fg	shap_bg	shap_fg
0	0.822	0.100	0.078	0.894	0.106
1	0.745	0.069	0.103	0.860	0.140
2	0.813	0.125	0.069	0.867	0.133
3	0.732	0.059	0.075	0.880	0.120
4	0.758	0.047	0.066	0.839	0.161

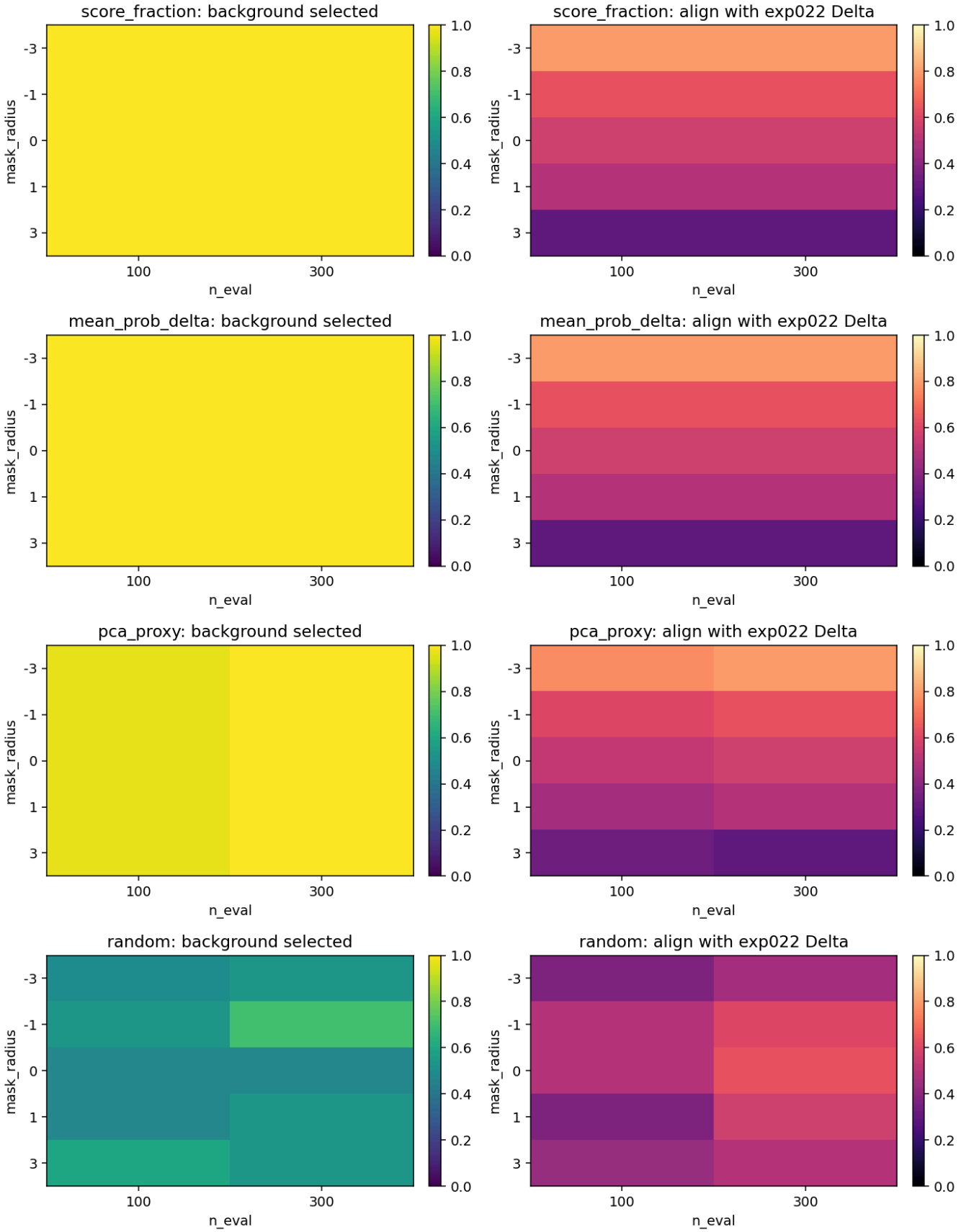
Waterbirds で両手法の判定が一致するのは、背景と前景という空間的に局在する二領域に対して、SHAP の空間集約が ColoredMNIST のような排他的符号化の影響を受けにくく、入力空間の構造を比較的素直に反映するためと考えられる。

候補発見と確認の接続評価

以下の接続評価は補助的な探索分析であり、GridWorld の sweep は 5 シード、CLIP / 自己教師あり表現の比較は 10 シードに基づく。まず GridWorld の候補発見 sweep では、低相関条件 `spurious_p=0.05` で `linear_probe_drop` が noise=7 の設定において top-1 4/5 (80%, 95% CI [0.38, 0.96])、noise=15 では 5/5 (100%, 95% CI [0.57, 1.00]) を示した一方、`spurious_p=0.25` では同順に 1/5 (20%, 95% CI [0.04, 0.62]) と 2/5 (40%, 95% CI [0.12, 0.77]) まで低下した。`mutual_info` と `tree_probe_drop` も同様に、top-k では真の候補を維持しつつ、top-1 回収は相関条件に応じて不安定になった。

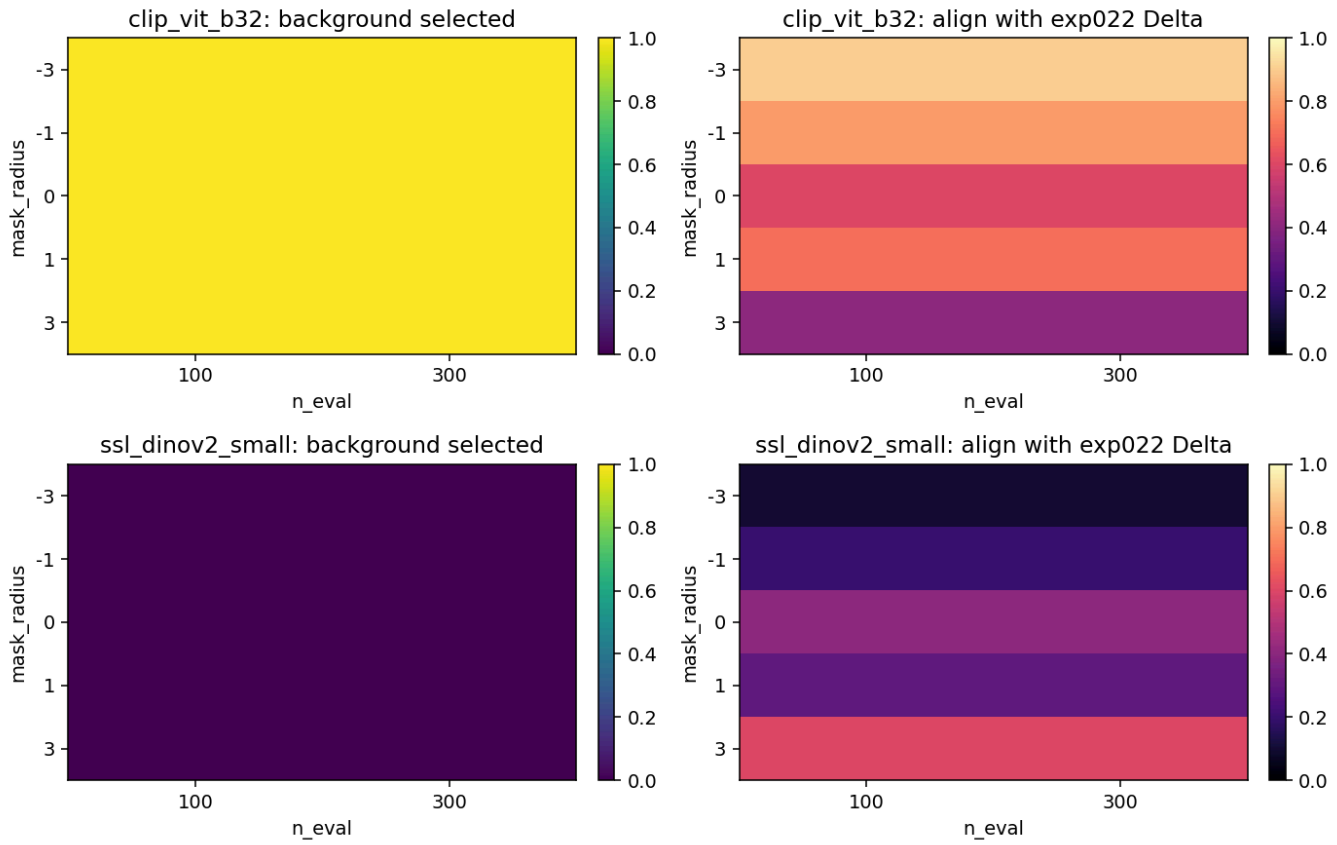


Waterbirds の候補発見 sweep では、単純なスコア比と平均確率差が 30 シード設定で背景を top-1 に 100% 置いた一方、PerformanceCheck 参照判定との整合率は半径 -3 で 24/30 (80%, 95% CI [0.63, 0.90])、半径 0 で 17/30 (57%, 95% CI [0.39, 0.73])、半径 3 で 9/30 (30%, 95% CI [0.17, 0.48]) であった。隠れ層主成分に基づく候補発見も 30 シード設定で top-1 97-100% を示したが、整合率は同様に半径依存であった。



さらに CLIP と自己教師あり表現を用いた候補発見では、CLIP ベースの候補発見が 10 シード全条件で背景を top-1 に置き、背景 top-1 は 10/10 (95% CI [0.72, 1.00])、参照判定との整合率は半径 -3, -1, 0, 1, 3 でそれぞれ 9/10 (90%, 95% CI [0.60, 0.98])、8/10 (80%, 95% CI [0.49, 0.94])、6/10 (60%, 95% CI [0.31, 0.83])、7/10 (70%, 95% CI [0.40, 0.89])、4/10 (40%, 95% CI [0.17, 0.69]) であった。これに対し自己教師あり表現は全条件で前景を top-1 に置き、背景 top-1 は 0/10 (95% CI [0.00, 0.28])、同じ順に 1/10 (10%, 95% CI [0.02, 0.40])、2/10 (20%,

95% CI [0.06, 0.51])、4/10 (40%, 95% CI [0.17, 0.69])、3/10 (30%, 95% CI [0.11, 0.60])、6/10 (60%, 95% CI [0.31, 0.83]) の整合率を示した。



考察

GridWorld の比較結果は、性能ベースの介入と出力分散ベースの寄与度推定とで、スプリアス特徴の検出能力に差が生じうることを示している。SHAP の Shapley 値は、特徴の「無効化」を訓練分布上の周辺化として定義するため、スプリアス特徴 c より因果特徴 agent_y の方が出力分散への寄与が大きく見える状況では、依存量を反映しにくくなる [Janzing2020, Adebayo2022]。PerformanceCheck は、特徴を破壊した後の性能を直接観測するため、出力分散寄与と独立に機能的必要性の近似指標を与える。GridWorld の二値的なスプリアス信号は、まさにこの差が顕在化しやすい設定であった。

実運用の観点では、最大値を与える一つだけを選ぶ使い方よりも、上位 k 件の候補を提示する使い方が適切な場合がある。中間的な相関設定では 1 位と 2 位の差が小さいケースも含まれるため、特徴数が多い設定では、Δ 値の高い上位 2〜3 特徴をドメイン専門家に提示し、スプリアス候補かどうかの判断を委ねる流れが現実的である。SHAP との比較においても、上位候補の絞り込みという観点で PerformanceCheck の優位は維持される。

ColoredMNIST における SHAP の見かけ上の満点は、データ符号化が寄与度集約に対して強い構造的制約を課すことに由来する。R と G の排他的符号化のもとでは、本稿で採用した color_attr = mean(|shap_R − shap_G|) と struct_attr = mean(|shap_R + shap_G|) / 2 という集約は、モデルの依存先によらず color_attr ≈ 2 × struct_attr を充足する。p=0.3 におけるシード別の挙動は、PerformanceCheck がモデルの構造依存への切り替えを反映しているのに対し、SHAP は同じ符号化制約のもとで一律に色依存を報告することを示す。判定スコア上は SHAP が「正解」と見えても、それは依存検出という意図に対して、性能低下を判断材料にしていない集約規則がほぼ自動的に一定方向の判定を返しているにすぎない。この観察は、特定のデータ符号化と SHAP

集約の組み合わせがアーティファクトを生む具体例として、寄与度推定法の適用時に注意すべき点を提供する[Adebayo2022]。

Waterbirds では、空間的に局在するスプリアス領域に対して SHAP の空間集約と Δ の介入が同じ背景候補を指しやすい一方、その候補が実際にどの程度の性能低下を引き起こすかについては PerformanceCheck がより保守的に選別している。背景と前景はピクセル空間で物理的に分離可能であり、SHAP 値の空間集約も Δ の介入も、共通の前景マスクという外部情報を利用できる。実世界スプリアスの一例として、候補発見と確認が補完的に働く状況を示している。

Waterbirds の専用ベースラインも、背景と前景のどちらがスプリアスかという粗い判断では Δ / SHAP と整合的だった。一方で、スコアの定義は hidden-space の統計量に依存しており、どの入力がどれだけの性能低下を引き起こすかをそのまま表すわけではない。したがって、専用ベースラインは「背景依存の候補確認」としては有用だが、運用上の必要性を直接測る指標としては Δ と役割が異なる。

Waterbirds の議論としては、専用ベースラインの「背景はスプリアス」という判定と、Δ の「その背景を壊すと実際に性能が落ちる」という判定は、競合ではなく役割分担として読むべきである。図が示すのは、候補確認の層では背景が一貫してスプリアスに見える一方で、介入後性能の層では一部のシードでそれが必ずしも致命的ではない、ということだ。18/30 にとどまった残りの 12/30 については、単一原因よりも、モデル学習のシード差、マスク境界の揺れ、そして背景優位か前景優位かの二値判定に圧縮した recall@1 定義の厳しさが重なっているとみるのが自然である。この意味で PerformanceCheck は、背景候補を広く警告する前段の候補発見に対し、性能低下を伴わない候補をはねることで誤警報を減らす、より保守的な確認段階として読める。

Waterbirds で候補発見の近似指標を前段に置いた追加実験も、この役割分担を支持していた。Results 節の接続評価が示すように、前段は「どの候補が疑わしいか」を返す一方、その向き自体が近似指標や表現空間に依存しうる。後段の PerformanceCheck は、その候補を実際に壊したとき性能がどの程度失われるかを測ることで、候補の存在と機能的必要性を切り分ける。Waterbirds のように候補集合が事実上 background / foreground の二択に近い設定では、候補発見が返した top-1 候補をそのまま後段へ渡して確認する半自動パイプラインは、実質的にはすでに構成できていると考えられる。今後の課題は、この流れを候補数の多い、より大きな環境へどこまで拡張できるかである。

CLIP と自己教師あり表現の差に対するひとつのもっともらしい解釈は、CLIP が `water` / `land` のような高水準の場面意味を、言語監督つきの事前学習を通じて比較的明示的に分離しており、それが Waterbirds ではラベルと強く相関しているため、前景置換より背景置換の方が表現変化を大きく生みやすかった、というものである。その結果、CLIP ベースの候補発見は背景を一貫して上位に置いたと考えられる。これに対し、本稿で用いた自己教師あり表現は、ラベル語彙に直接結びつかない表現空間で対象中心的な構造や局所視覚特徴を強く保つため、鳥本体側の変化により敏感であり、異なる候補順位を返した可能性がある。実際、CLIP では背景 top-1 が 10/10 で整合率も最大 9/10 まで達したのに対し、自己教師あり表現では背景 top-1 が 0/10 で、整合率は背景確認が弱い条件ほど上がるという逆向きの振る舞いを示した。この差は、候補発見が表現空間に依存することを示しており、後段で機能的必要性を確認する PerformanceCheck の役割を補強する。

PerformanceCheck は、専門家がデータ拡張、背景置換、追加データ収集の優先順位を決めるための定量的根拠を与える部品として位置づけるのが自然である。発見手法や SHAP / PCA のような近似指標で疑わしい候補を挙げ、その後に PerformanceCheck で実際に性能を壊すかどうかを確認し、最後に Δ の大きい候補から修正の優先順位を付ける、という三段の流れが実務上は扱いやすい。Waterbirds で背景候補が高順位に残り、かつ専用ベースラインや CLIP 系の候補発見でも同じ候補が繰り返し上がるなら、まず背景置換によるデータ拡張や背景バリエーションの追加収集を優先し、それでも Δ が残る場合に再学習やモデル選択の変更を検討する、という順序が自然である。

worst-group accuracy の再集計は、この Waterbirds のモデルが平均精度だけでは十分に見えにくい群間不均衡を抱えていることを示す。特に最悪群の性能が低い場合、背景依存の検出が「全体として当たっているか」ではなく、「どの群で壊れているか」を見る補助情報になる。Waterbirds の Δ / SHAP 比較と worst-group 指標を併せることで、背景スプリアス依存の有無と、その依存が最悪群の性能低下に結びついているかを分けて読める。

特徴数が大きい設定への拡張については、全候補への介入による総当たり評価を基本線としつつ、計算制約が厳しい場合に限って「候補絞り込み後介入」を補助的に使う。具体的には、発見手法（Neuhaus ら [Neuhaus2023]、Eyuboglu ら [Eyuboglu2022]）または寄与度推定法で上位 k 特徴を抽出し、その集合内で Δ を評価する。ただし本研究の結果が示す通り、寄与度推定法単体の順位は依存検出と乖離しうるため、絞り込みは計算上の近似であり、主張の根拠は最終的に介入後性能差分で与える。

PerformanceCheck と既存の寄与度推定法は、目的の違いから条件付きで補完的に使い分けられる。SHAP や統合勾配は説明のため、PerformanceCheck は性能感度の監査のために向く。

限界

本研究にはいくつかの限界がある。まず、データと頑健性の観点では、GridWorld / ColoredMNIST / Waterbirds を 30 シードで評価したものの、より大規模な seed 数や別学習器でも同様の傾向が維持されるかは今後の課題である。Waterbirds の凍結 ResNet-18 fine-tune 訓練済みモデルも検証精度が 0.759 から 0.822 の範囲にあり、より高精度な設定で同じ検出率が保たれるかは確認していない。また、ColoredMNIST における SHAP のアーティファクト分析は本研究で採用した R-G 非対称集約に固有であり、別の集約方式や別の寄与度推定法では異なる挙動が観察されうる。本研究の指摘は、特定の集約方式を採用したとき構造的に何が起きるかを示すものであり、SHAP 一般の不能を主張するものではない。

次に、手法の一般性の観点では、介入演算子の設計は問題ごとに人手で行っており、自動化されていない。GridWorld のように特徴次元が分離可能な設定では周辺リサンプリングが自然だが、Waterbirds の前景マスクは GrabCut という外部アルゴリズムに依存する。介入の質は判定結果に直接影響するため、介入設計の標準化は実用上の課題である。具体的には、背景置換が背景手掛かりを十分に破壊できているか、同時に鳥本体の輪郭や局所テクスチャをどの程度保存できているかを定量的に測る補助指標は、今後さらに必要になる。目視確認や、前景保持率・背景置換率のような補助統計を併用して、介入がどこまで対象特徴だけを壊しているかを点検する仕組みも望ましい。PerformanceCheck が比較的素直に解釈できるのは、GridWorld の特徴次元や Waterbirds の背景領域のように、候補特徴を局所化または分離して介入しやすい場合であり、候補特徴が他特徴と強く絡み合う設定では OOD 由来の性能低下との切り分けが難しくなる。さらに、検出判定は候補特徴が事前に列挙されている設定を前提としており、候補の自動列挙自体は Neuhaus ら [Neuhaus2023]、Eyuboglu ら [Eyuboglu2022]、Sohoni ら [Sohoni2020] などの既存研究に依存する。本稿が主に扱うのも候補列挙の後から Δ を計測する段階までであり、「どの特徴をどのようにシフトすべきか」という設計問題自体は別段の課題である。Waterbirds の class-consistent 背景置換も、強い OOD 介入を避ける代わりに、クラスと背景が結び付いた偏りを部分的に残す可能性があり、クラス非依存な背景バイアスには鈍感になりうる。計算量も全候補への介入による総当たり評価では概ね候補数と評価サンプル数の積に比例するため、高次元設定では候補絞り込み後介入が必要になるが、その場合は候補落ちによる取りこぼしの危険も増える。加えて、RL については本稿では正例をまだ示せておらず、LunarLander の結果は適用境界の確認にとどまる。Δ の絶対値そのものに対する閾値設計（依存ありとなしの境界）は本研究では行わず、effect size を含む追加の統計整理も今後の補強余地として残る。今後は、介入設計の標準化と自動化、および候補発見法と組み合わせた、より統合的な一貫自動評価を進める必要がある。

社会的影響

PerformanceCheck は、訓練後のモデルに対するスプリアス依存の診断を、入力空間での介入と性能観測のみで実施できるため、デプロイ前の安全性検査や信頼性評価のツールとして用いうる。たとえば、背景依存が安全性や公平性の問題に直結しうる医療画像や自動運転用視覚モデルの監査では、候補特徴を壊したときに本当に性能が落ちるかを確認する手続きとして有用である。一方で、検出能力には適用前提と介入設計の質が直接影響するため、診断結果を過度に一般化して「依存なし」と判定するリスクがある。ColoredMNIST における SHAP のアーティファクト事例は、ベンチマーク設計とデータ符号化の組み合わせが寄与度推定法の判定をほぼ自動的に決めうることを示しており、診断結果のみに基づく意思決定には慎重な検証が必要である。本研究の知見は、複数の診断手法を併用し、各手法の前提条件を明示することの重要性を支持する。

結論

本研究の主要な結論は、発見済みのスプリアス候補に対して、破壊的介入による性能感度 Δ で「本当に性能に効いているか」を確認する後段手続きを定式化したことである。GridWorld では PerformanceCheck が top-1 85/120 (71%, 95% CI [0.62, 0.78])、top-2 106/120 (88%, 95% CI [0.81, 0.93]) を示し、SHAP より高い検出率を与えたうえ、対応のある McNemar 検定でも差が有意だった。ColoredMNIST IRM では PerformanceCheck が 102/120 (85%, 95% CI [0.78, 0.90])、SHAP が 120/120 (100%, 95% CI [0.97, 1.00]) を示したが、その差は本稿で採用した集約規則の下での構造的制約として説明でき、対応のある McNemar 検定でも差が確認された。Waterbirds では SHAP が 30/30 (100%, 95% CI [0.89, 1.00]) で背景候補を指摘し、PerformanceCheck は 18/30 (60%, 95% CI [0.42, 0.75]) で背景介入の方が前景介入より大きな性能差分を与えることを示した。さらに GrabCut マスク半径の感度分析、および専用スコア、CLIP、自己教師あり表現を用いた候補発見実験は、候補の存在を拾う前段と、性能上の必要性を確認する後段とが異なる役割を持つことを支持した。Waterbirds のように候補集合が小さい設定では、この後段確認を組み込んだ半自動パイプラインは事実上ほぼ構成できている。以上より、PerformanceCheck は候補発見の代替ではなく、その後段で候補の機能的必要性を確認し、介入ベース診断を実用段階へ押し出す情報を与える手続きとして有用である。

付録

A. LunarLander 実験の詳細

LunarLander-v2 環境において PPO で訓練した訓練済み方策に対し、候補スプリアス特徴（着地脚接触フラグ等）を置換する介入を試みた。複数の spurious_p 設定および候補特徴設定で評価したが、いずれの設定でも訓練済み方策が候補特徴を利用せず、Δ ≈ 0 であった。追加のアブレーションでは、報酬の割引率に関連する信号減衰が訓練済み方策の学習を困難にしていることが示唆された。本稿 Limitations 節で述べた通り、これらの結果は PerformanceCheck の失敗ではなく、検出対象（スプリアス依存）が成立しない設定に該当する。

B. ハイパーパラメータ詳細

GridWorld

パラメータ	値
アルゴリズム	DQN (stable-baselines3)
総ステップ数	50,000

パラメータ	値
学習率	1e-4（デフォルト）
バッファサイズ	50,000
バッチサイズ	32
ε-greedy 最終値	0.05
サロゲート（MoET）	4 experts, max_depth=5
蒸留エポック数	100
蒸留サンプル数	10,000
SHAP 背景サンプル数	100
SHAP 評価サンプル数	300

ColoredMNIST IRM

パラメータ	値
アーキテクチャ	CNN (Conv×2 + FC×2)
エポック数	20
バッチサイズ	64
学習率	1e-3
IRM ペナルティ係数 λ	1.0（オリジナル設定準拠）
label_noise_p	0.25
SHAP 背景サンプル数	100
SHAP 評価サンプル数	200

Waterbirds

パラメータ	値
アーキテクチャ	ResNet-18（ImageNet事前学習済み、特徴抽出層凍結）
分類層	Linear(512, 2)
エポック数	10
バッチサイズ	32
学習率	1e-3
最適化器	Adam
前景マスク	GrabCut（OpenCV実装）
背景ノイズ標準偏差	0.3

パラメータ	値
SHAP 背景サンプル数	50
SHAP 評価サンプル数	100

C. 計算資源

全実験は CPU のみで実施した（GPU なし）。

実験	環境	1シードあたり所要時間
GridWorld（DQN + MoET + SHAP）	AMD Ryzen 7 5700X, 8コア	約 15 分
ColoredMNIST（CNN + IRM + SHAP）	同上	約 20 分
Waterbirds（ResNet-18 + SHAP）	同上	約 46 分

D. 再現性情報

- コード: 本稿の実験コードは `rl-shift-auto-detection` リポジトリとして管理し、最終版を公開予定。
- データセット:
  - ColoredMNIST: torchvision MNIST をもとに論文設定（label_noise_p=0.25）で生成。
  - Waterbirds: `waterbird_complete95_forest2water2` サブセットを使用（metadata.csv に従う）。
- 実行環境:
  - Python 3.13
  - 主ライブラリ: PyTorch, torchvision, stable-baselines3, shap, pandas, numpy
- 乱数制御:
  - numpy / PyTorch / 介入サンプリングでシードを固定。
  - Δ と SHAP 比較は同一の訓練済みモデル・同一シードで実施。
  - seed 一覧は GridWorld / ColoredMNIST / Waterbirds のすべてで `0,1,2,...,29` を用いた。
  - Waterbirds では class-consistent な候補群を各クラス 200 枚で構成し、`background_replace` は `n_eval=300` 回、`foreground_noise` は標準偏差 `0.3` で評価した。

主要結果の再実行に用いたコマンドと設定ファイルは次の通りである。原稿時点の参照コミットは `9b84f2d` とする。

実験	実行コマンド
GridWorld p-sweep	`python experiments/run_gridworld_psweep.py --config experiments/configs/exp005_gridworld_psweep.yaml --outdir experiments/results/exp005_gridworld_psweep`
GridWorld Δ vs SHAP	`python experiments/run_shap_psweep.py --config experiments/configs/exp016_shap_psweep_30seeds.yaml --outdir experiments/results/exp016_shap_psweep_30seeds`

実験	実行コマンド
ColoredMNIST Δ vs SHAP	`python experiments/run_cmnist_shap_psweep.py --config experiments/configs/exp017_cmnist_shap_psweep_30seeds.yaml --outdir experiments/results/exp017_cmnist_shap_psweep_30seeds`
Waterbirds Δ vs SHAP	`python experiments/run_waterbirds.py --config experiments/configs/exp018_waterbirds_30seeds.yaml --outdir experiments/results/exp018_waterbirds_30seeds`
Waterbirds worst-group accuracy	`python experiments/compute_waterbirds_wga.py --input experiments/results/exp018_waterbirds_30seeds/results.csv --outdir experiments/results/exp019_waterbirds_wga`
Waterbirds マスク感度	`python experiments/run_waterbirds_grabcut_sensitivity.py --config experiments/configs/exp022_waterbirds_grabcut_sensitivity.yaml --outdir experiments/results/exp022_waterbirds_grabcut_sensitivity`
GridWorld 候補発見 sweep	`python experiments/run_gridworld_candidate_discovery_noise_p_sweep.py --config experiments/configs/exp024_gridworld_candidate_discovery_noise_p_sweep.yaml --outdir experiments/results/exp024_gridworld_candidate_discovery_noise_p_sweep`
Waterbirds 候補発見 sweep	`python experiments/run_waterbirds_candidate_discovery_sweep.py --config experiments/configs/exp025_waterbirds_candidate_discovery_sweep.yaml --outdir experiments/results/exp025_waterbirds_candidate_discovery_sweep`
Waterbirds CLIP / 自己教師あり表現	`python experiments/run_waterbirds_representation_candidate_discovery.py --config experiments/configs/exp026_waterbirds_clip_ssl_candidate_discovery.yaml --outdir experiments/results/exp026_waterbirds_clip_ssl_candidate_discovery`

References

[Adebayo2022] Adebayo, J., et al. "Post hoc Explanations may be Ineffective for Detecting Unknown Spurious Correlation." ICLR 2022.

[Arjovsky2019] Arjovsky, M., et al. "Invariant Risk Minimization." arXiv:1907.02893, 2019.

[Eyuboglu2022] Eyuboglu, S., et al. "Domino: Discovering Systematic Errors with Cross-Modal Embeddings." ICLR 2022.

[Geirhos2020] Geirhos, R., et al. "Shortcut Learning in Deep Neural Networks." Nature Machine Intelligence, 2020.

[He2016] He, K., et al. "Deep Residual Learning for Image Recognition." CVPR 2016.

[Janzing2020] Janzing, D., et al. "Feature relevance quantification in explainable AI: A causal problem." AISTATS 2020.

[Lapuschkin2019] Lapuschkin, S., et al. "Unmasking Clever Hans Predictors and Assessing What Machines Really Learn." Nature Communications, 2019.

[Lundberg2017] Lundberg, S. M., and Lee, S.-I. "A Unified Approach to Interpreting Model Predictions." NeurIPS 2017.

[Neuhaus2023] Neuhaus, Y., et al. "Spurious Features Everywhere — Large-Scale Detection of Harmful Spurious Features in ImageNet." ICCV 2023.

[Pearl2000] Pearl, J. "Causality: Models, Reasoning, and Inference." Cambridge University Press, 2000.

[Ribeiro2016] Ribeiro, M. T., et al. "Why Should I Trust You? Explaining the Predictions of Any Classifier." KDD 2016.

[Sagawa2020] Sagawa, S., et al. "Distributionally Robust Neural Networks for Group Shifts." ICLR 2020.

[Singla2022] Singla, S., and Feizi, S. "Core Risk Minimization using Salient ImageNet." ICML 2022.

[Sohoni2020] Sohoni, N., et al. "No Subclass Left Behind: Fine-Grained Robustness in Coarse-Grained Classification Problems." NeurIPS 2020.

[Sundararajan2017] Sundararajan, M., et al. "Axiomatic Attribution for Deep Networks." ICML 2017.

[Veitch2021] Veitch, V., et al. "Counterfactual Invariance to Spurious Correlations in Text Classification." NeurIPS 2021.

[Wah2011] Wah, C., et al. "The Caltech-UCSD Birds-200-2011 Dataset." Technical Report, 2011.

[Zhou2021] Zhou, C., et al. "Examining and Combating Spurious Features under Distribution Shift." ICML 2021.